

# CS 184: Computer Graphics and Imaging, Spring 2018

## Project 2: Mesh Edit

Morgan Lyu

### Overview

In this project, methods to generate Bezier curves and surfaces using the de Casteljau algorithm was implemented. Area-weighted normals are implemented to make the mesh smoother. Ways to flip and split mesh edges by using half-edge data structure were also implemented, and functionality to smoothen angular features with loop subdivision was added.

### Section I: Bezier Curves and Surfaces

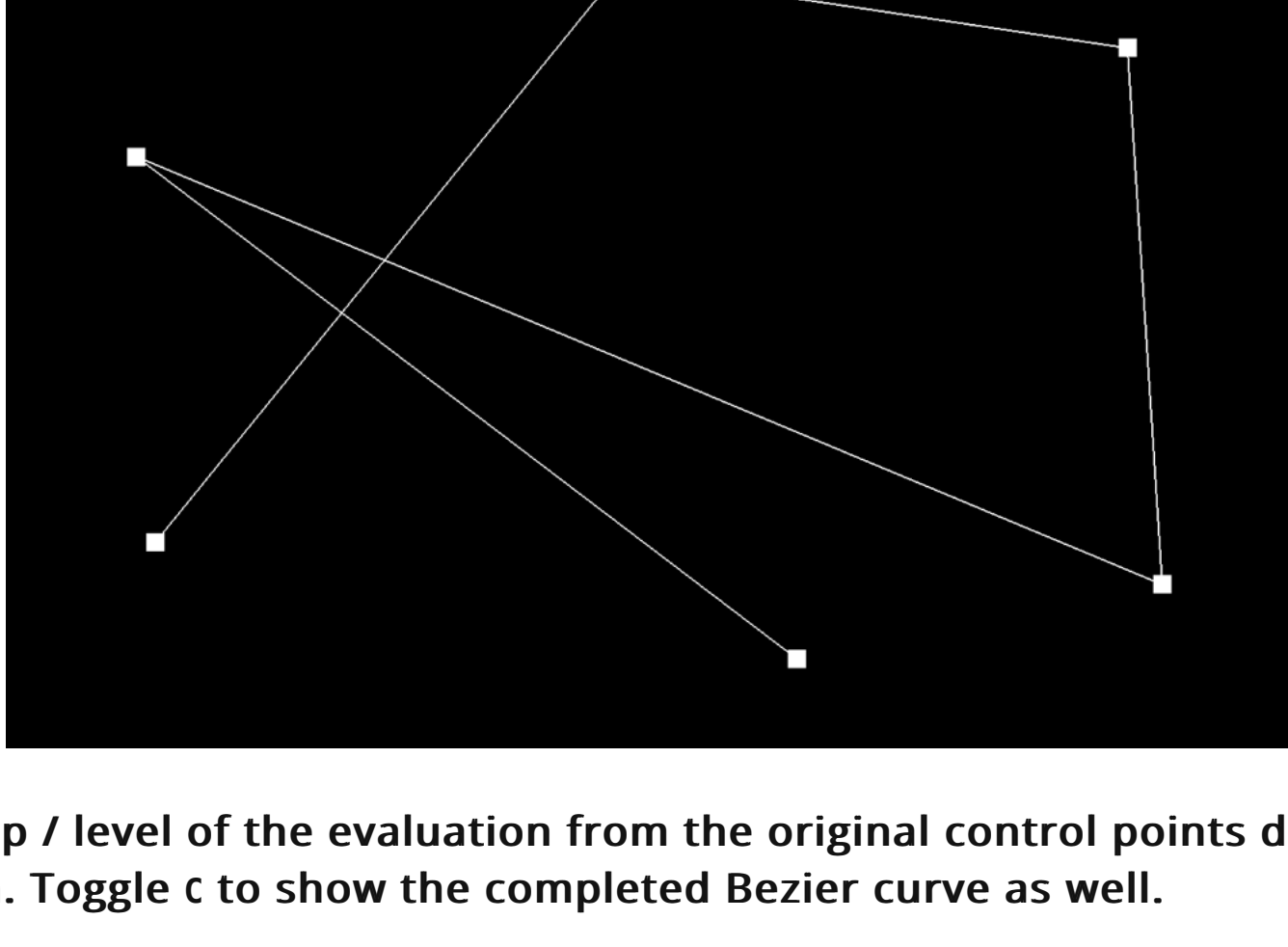
#### Part 1: Bezier Curves with 1D de Casteljau Subdivision

Briefly explain de Casteljau's algorithm and how you implemented it in order to evaluate Bezier curves.

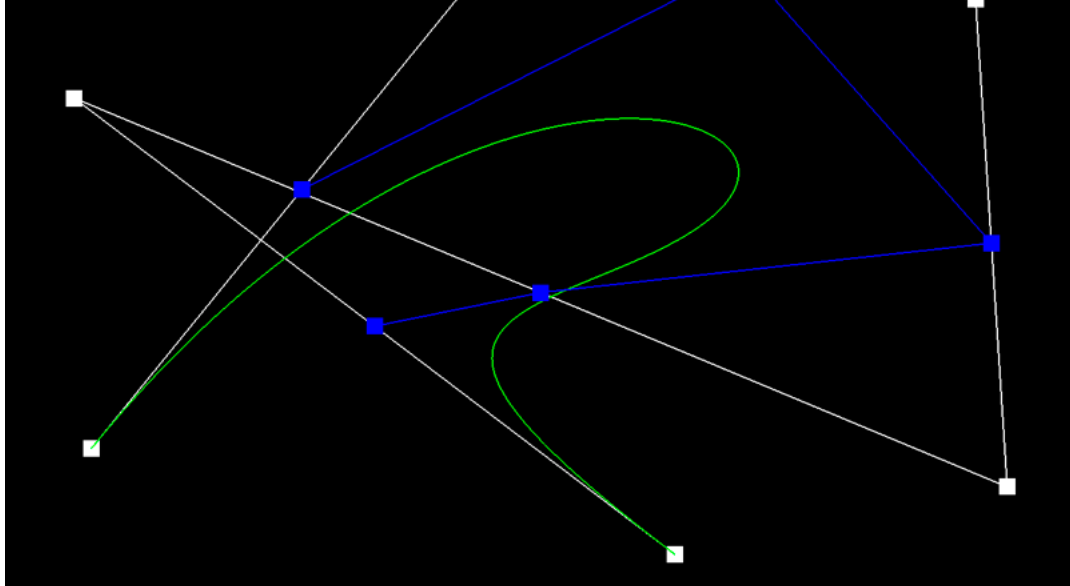
The algorithm works by performing linear interpolation for N control point's N-1 edges and obtaining N-1 new control points for a given parameter t, a ratio specifying where the point occurs on a given edge. The exact position is calculated by  $(1-t) * p_1 + t * p_2$ , where p1 and p2 are point positions defining the edge. Then, the same process is conducted recursively for the N-1 control points until we have down to 1 point. This point is part of the bezier curve defined by the N control points. By adjusting t from 0 to 1, we can obtain all the points on the bezier curve.

Take a look at the provided .bzc files and create your own Bezier curve with 6 control points of your choosing. Use this Bezier curve for your screenshots below.

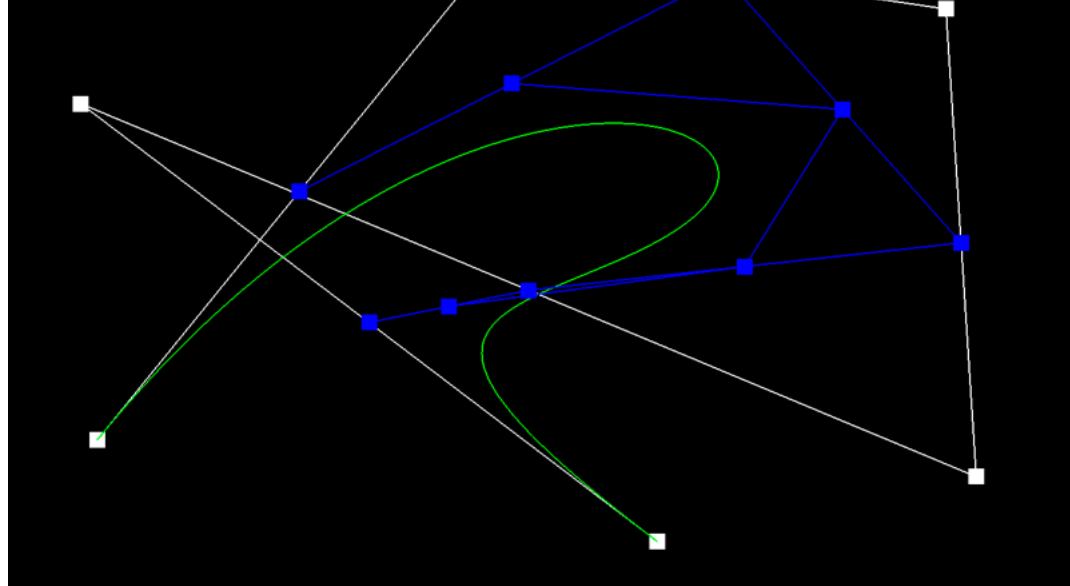
This file is located in the bzc folder titled curve3.



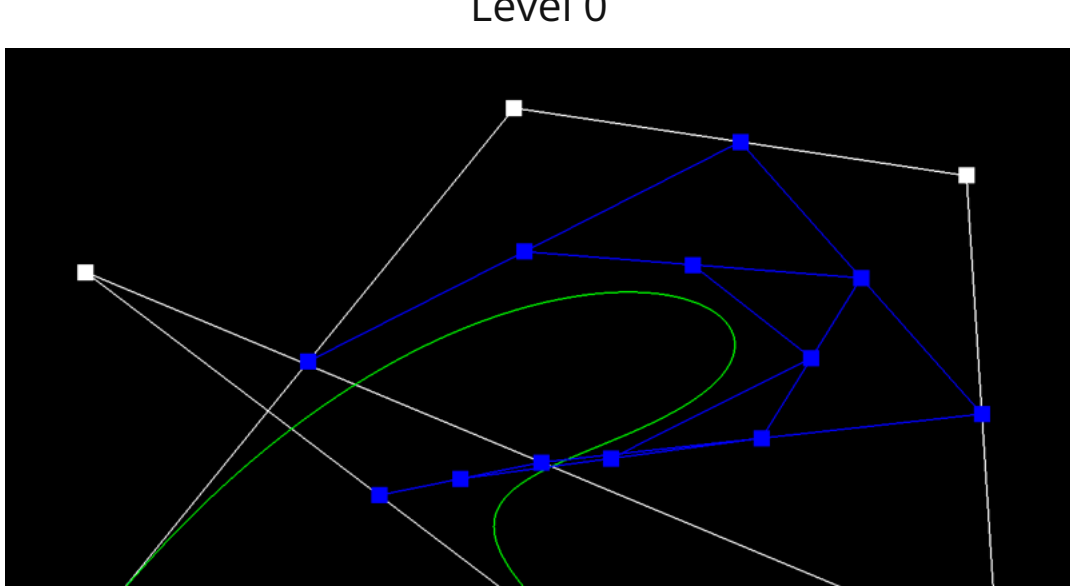
Show screenshots of each step / level of the evaluation from the original control points down to the final evaluated point. Press E to step through. Toggle C to show the completed Bezier curve as well.



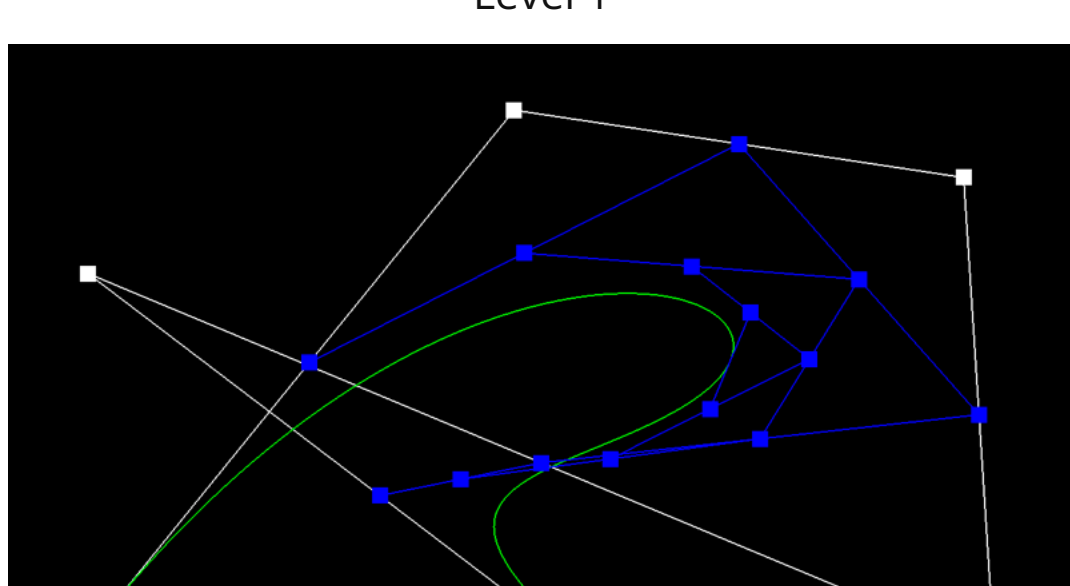
Level 0



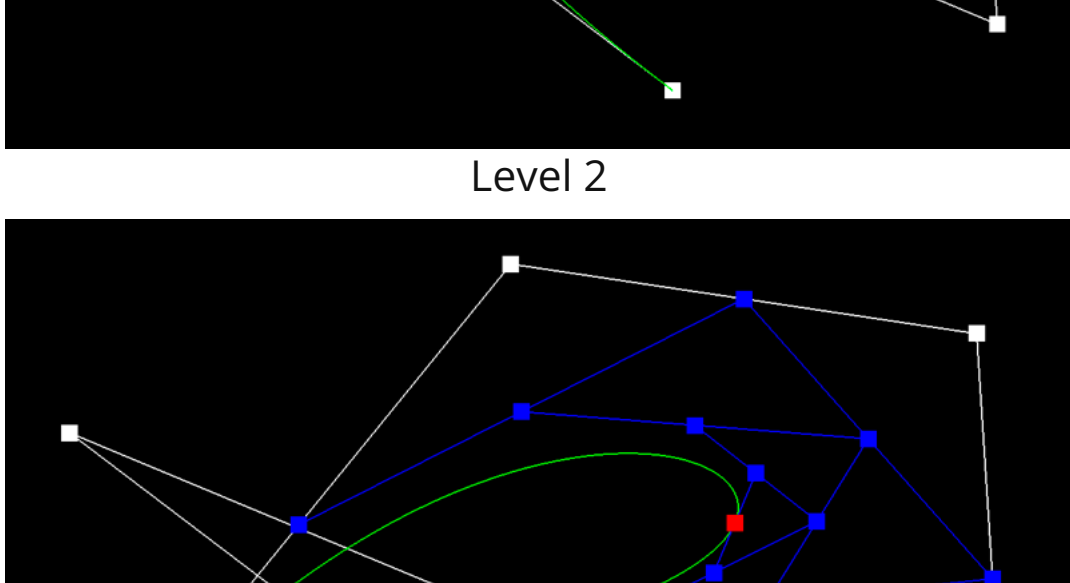
Level 1



Level 2

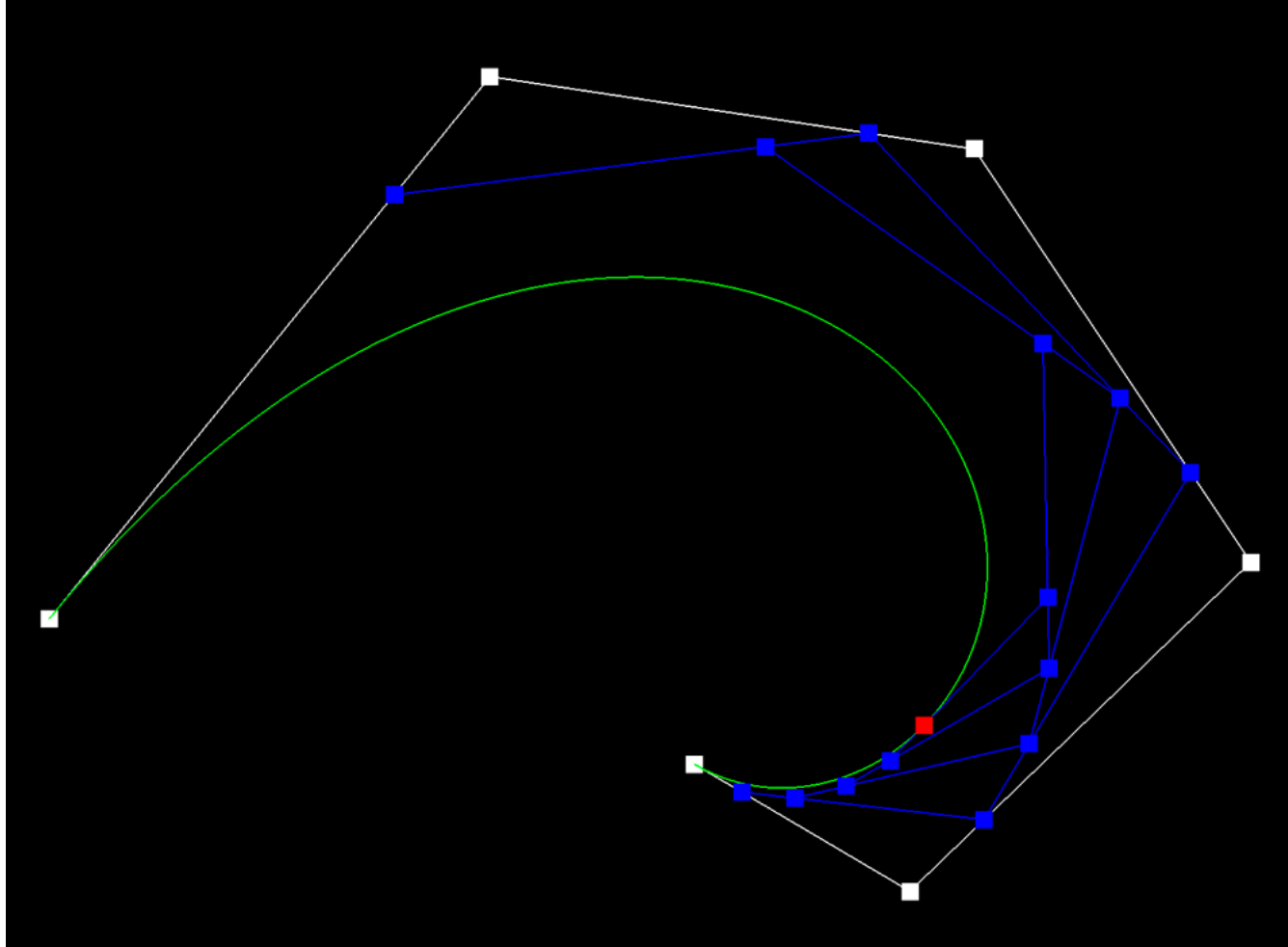


Level 3



Level 4

Show a screenshot of a slightly different Bezier curve by moving the original control points around and modifying the parameter t via mouse scrolling.

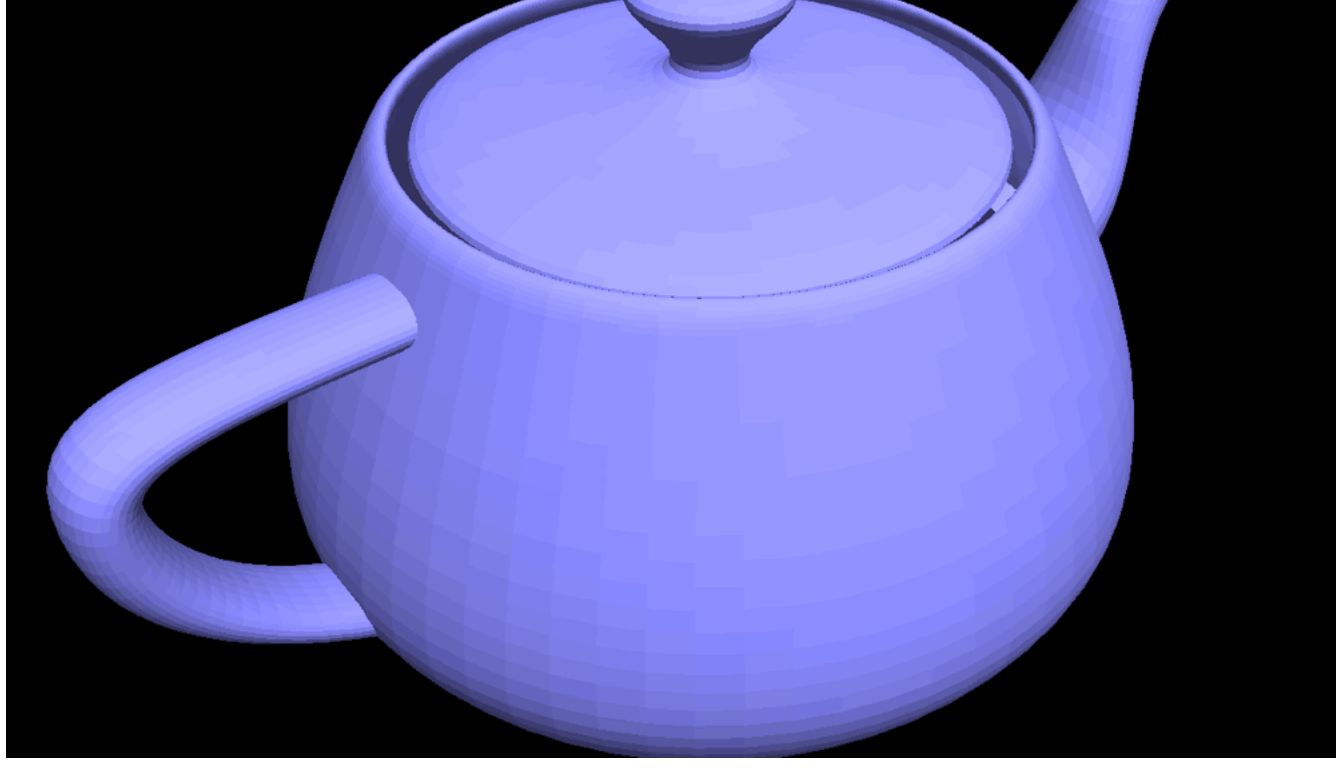


#### Part 2: Bezier Surfaces with Separable 1D de Casteljau

Briefly explain how de Casteljau algorithm extends to Bezier surfaces and how you implemented it in order to evaluate Bezier surfaces.

The algorithm applies to 3D Bezier surfaces the same way it does to 2D Bezier curves. We merely repeat the process for 1 more dimension, and instead of 1 linear parameter t, we have 2 (u and v) for 2 dimensions. My implementation first applies the Casteljau algorithm horizontally for each row in n rows using parameter u to obtain n final points. Then, these points are treated as a column of control points for another round of vertical Casteljau with parameter v. The single point obtained after this is on the Bezier surface. Adjusting u and v from 0 to 1, the entire Bezier surface is obtained.

Show a screenshot of bez/teapot.bez (not .dae) evaluated by your implementation.



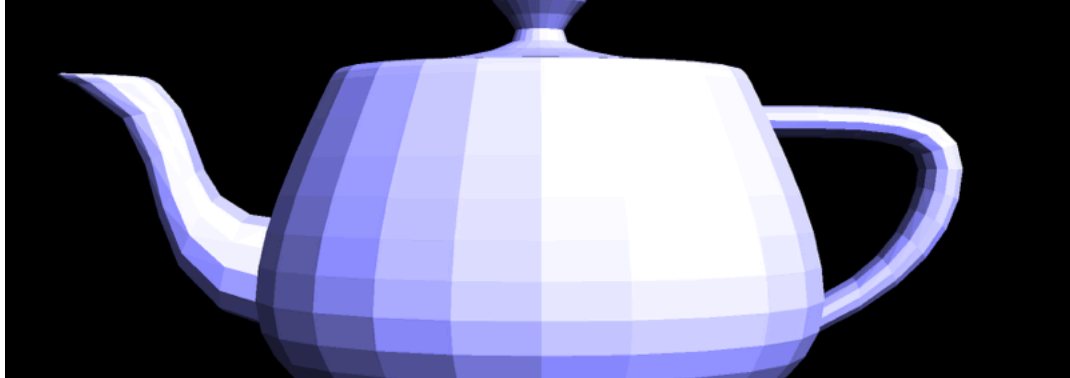
### Section II: Triangle Meshes and Half-Edge Data Structure

#### Part 3: Area-Weighted Vertex Normals

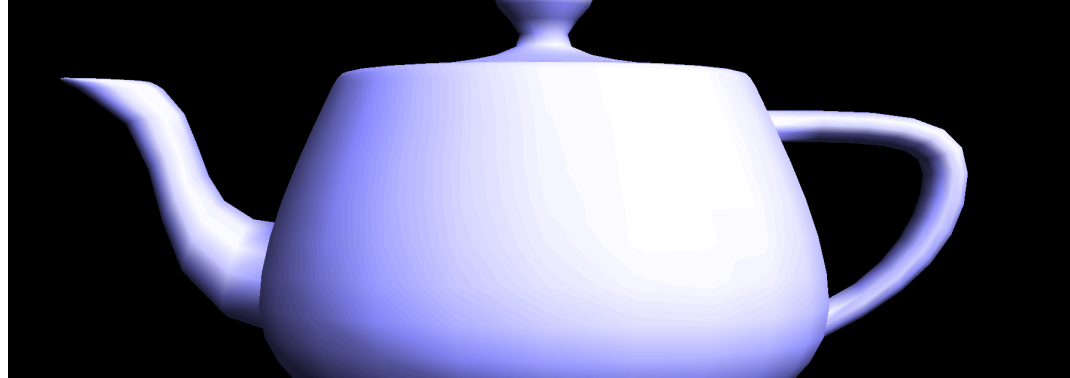
Briefly explain how you implemented the area-weighted vertex normals.

To find the normal for a vertex Va, I first found the areas of triangles surrounding the vertex. The magnitude of this area would be the magnitude of the normal for the vertex. To do find the areas for each triangle, I took cross products of vectors originating from Va to consecutive vertices neighboring it, and then divided it by 2.

Show screenshots of dae/teapot.dae (not .bez) comparing teapot shading with and without vertex normals. Use Q to toggle default flat shading and Phong shading.



No normals



With normals

#### Part 4: Edge Flip

Briefly explain how you implemented the edge flip operation and describe any interesting implementation / debugging tricks you have used.

To implement edge flip, mesh vertex, edge, face relations in the half-edge data structure is redefined in order to flip a given edge. Every element have a half-edge, and to modify their half-edge properties (next, twin, vertex, edge, face), next and twin is used to iterate through them all, and setNeighbors is used to reassign values. Drawing a picture of the geometry with elements labeled before and after the flip helped with debugging.

Show screenshots of the teapot before and after some edge flips.



Before



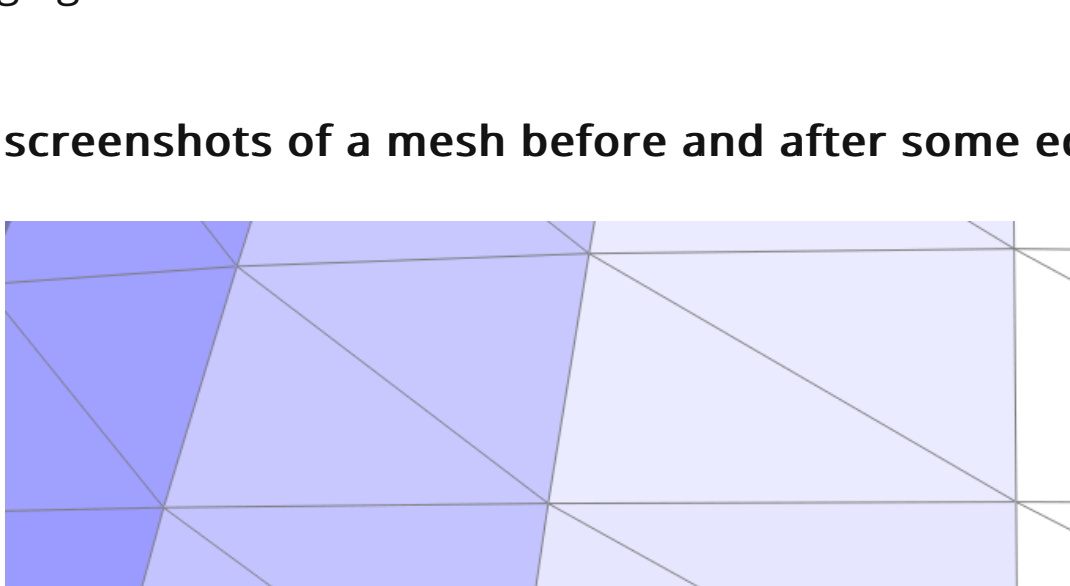
After

#### Part 5: Edge Split

Briefly explain how you implemented the edge split operation and describe any interesting implementation / debugging tricks you have used.

Compared to edge flip, edge split also features a reassignment of mesh element relationships. There are 6 more edges, 2 more faces, and 1 more vertex involved. Drawing a picture of the geometry with elements labeled before and after the split helped with debugging.

Show screenshots of a mesh before and after some edge splits.

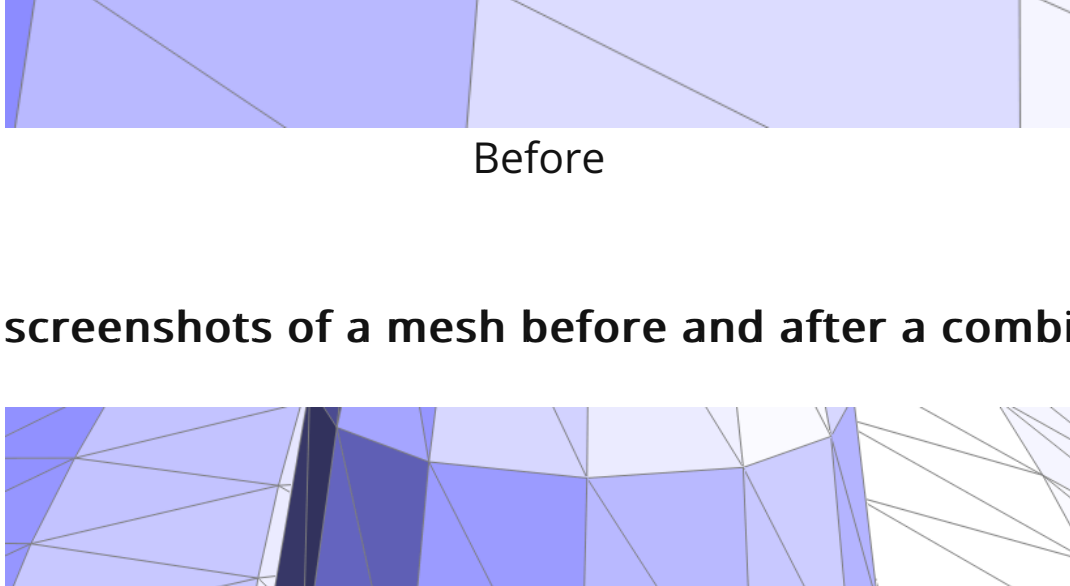


Before



After

Show screenshots of a mesh before and after a combination of both edge splits and edge flips.



Before



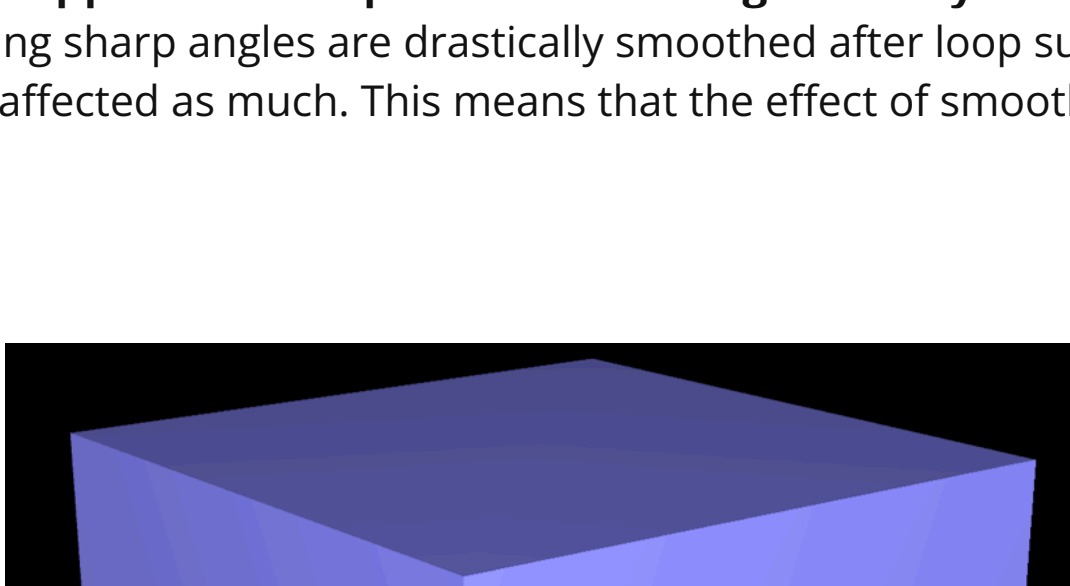
After

#### Part 6: Loop Subdivision for Mesh Upsampling

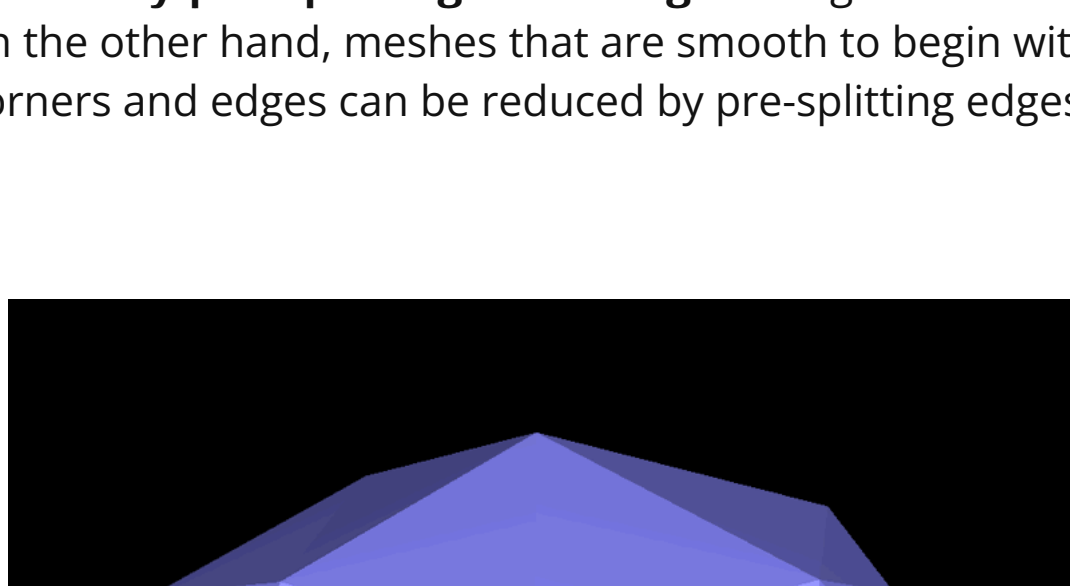
Briefly explain how you implemented the loop subdivision and describe any interesting implementation / debugging tricks you have used.

There are 5 steps. 1. All existing vertices have their field isNew set to false. New positions of these vertices are calculated according to equation given and stored in existing vertices' newPosition fields. 2. All existing edge have their field isNew set to false. New positions of these edges are calculated and stored in existing edges' newPosition fields. 3. All existing edges are split, new vertices are created and their new positions are determined based on a given weighting scheme. 4. Iterate through all edges (old and new) and flip the ones connecting an old vertex and a new vertex. 5. Iterate through all vertex (old and new) and update their position.

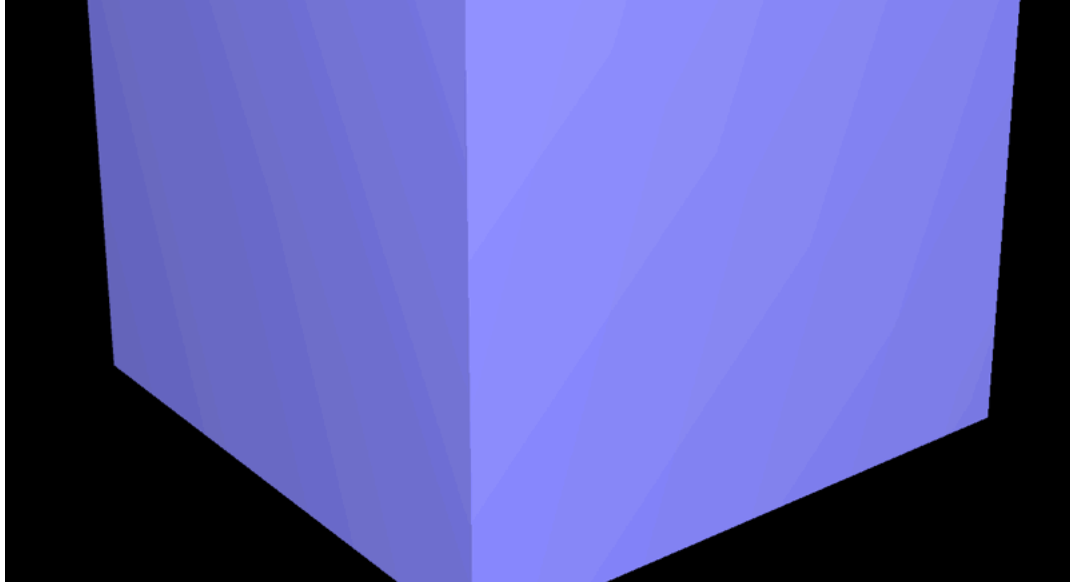
Take some notes, as well as some screenshots, of your observations on how meshes behave after loop subdivision. What happens to sharp corners and edges? Can you reduce this effect by pre-splitting some edges? Angular meshes subdivides symmetrically? Document these effects and explain why they occur. Also explain how your pre-processing helps alleviate the effects. It occurs because the edges going to vertices (degrees) are not equal: some have 4, some have 5. By splitting the diagonal edges across the cubes' 6 faces, this problem is resolved, as every vertex now have 6 edges going towards it.



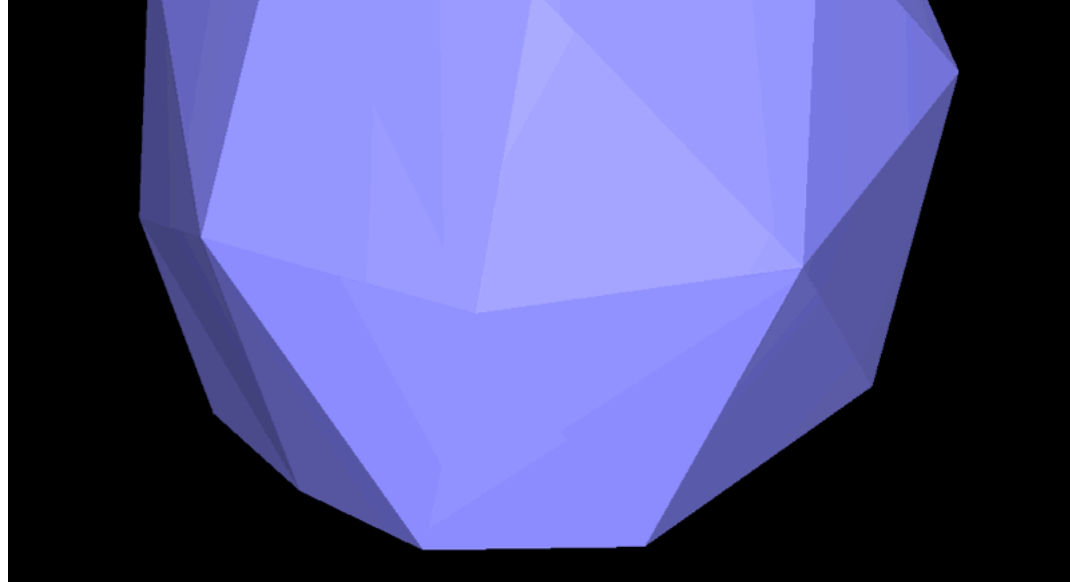
Cube



Subdivided Cube

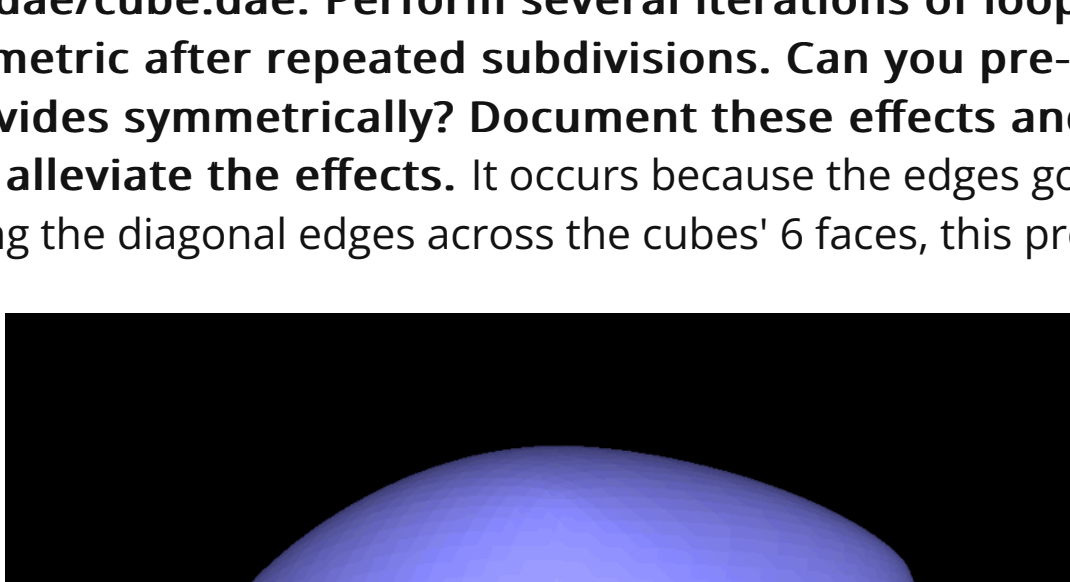


Bean

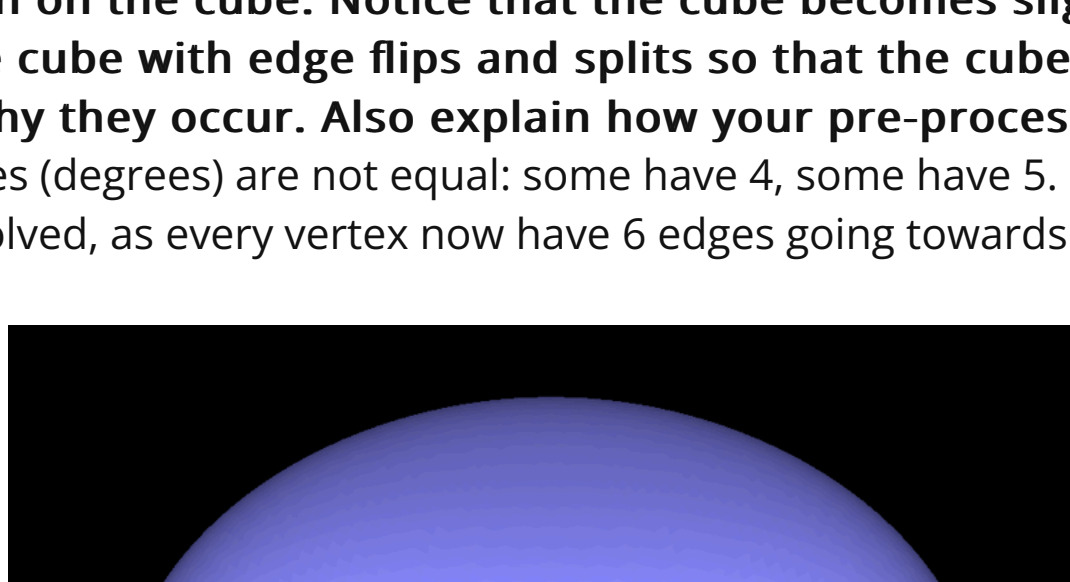


Subdivided Bean

Load dae/cube.dae. Perform several iterations of loop subdivision on the cube. Notice that the cube becomes slightly asymmetric after repeated subdivisions. Can you pre-process the cube with edge flips and splits so that the cube subdivides symmetrically? Document these effects and explain why they occur. Also explain how your pre-processing helps alleviate the effects. It occurs because the edges going to vertices (degrees) are not equal: some have 4, some have 5. By splitting the diagonal edges across the cubes' 6 faces, this problem is resolved, as every vertex now have 6 edges going towards it.



Before



After